

Reviewer Pack — Real Polynomial Approximation Degree Lower Bound for AC⁰ Cir...

Entry 4d43beaa3b30 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-19 06:45:10 UTC

Real Polynomial Approximation Degree Lower Bound for AC⁰ Circuits Computing PARITY

Entry ID: 4d43beaa3b30 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-19 06:45:10 UTC

1. Conjecture

Field A (mathematical branch): REAL_ALGEBRAIC_GEOMETRY **Field B** (complexity object): CIRCUIT_COMPLEXITY

Statement:

Any AC⁰ circuit computing PARITY on n inputs requires depth $\Omega(\sqrt{n})$, and the minimum degree of a real polynomial approximating PARITY with error $\varepsilon = 1/n$ is $\Omega(\sqrt{n})$, implying that the circuit’s size is exponential in depth.

Rationale (proposer’s reasoning):

The degree of a real polynomial approximating PARITY is tied to the circuit’s depth. Since AC⁰ circuits cannot compute PARITY exactly, their approximation degree must be high, leading to a lower bound on circuit size. This replaces the switching lemma with a geometric invariant under input permutations (symmetry of PARITY).

Taxonomy category: AC0_PARITY (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): cb42e9aefc878120

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

default: $\geq 80\%$ seeds must support, no counterexample

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.95	SAFE	SAFE
NATURAL_PROOFS	SAFE	0.95	UNCERTAIN	SAFE

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
ALGEBRIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.95	UNCERTAIN	SAFE

4. Novelty audit

Verdict: NOVEL against 2 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - AC⁰ circuits PARITY real polynomial approximation - AC⁰ PARITY lower bound real polynomial approximation - AC⁰ PARITY depth approximation degree real polynomial

Top relevant hits considered: - [s2:2502.15928] EHands: Quantum Protocol for Polynomial Computation on Real-Valued Encoded States - [s2:2411.00976] Low-degree approximation of QAC⁰ circuits

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=124, elapsed=246.4s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    n = 40
    if n < 5 or n > 40:
        return {
            "metric_name": "degree",
            "metric_value": None,
            "instances_tested": 0,
            "conjecture_holds": False,
            "counterexample": "n_out_of_range"
        }

    # Generate the parity function
    def parity(x):
        return sum(x) % 2

    # Generate random inputs
    inputs = [random.randint(0, 1) for _ in range(n)]
    target_value = parity(inputs)

    # Construct the polynomial approximation problem
    A = []
    b = []
    for i in range(2**n):
        x = [(i >> j) & 1 for j in range(n)]
        A.append(x + [parity(x)])
        b.append(target_value if x == inputs else 0)

    # Solve the linear system using Gaussian elimination
```

```

def gaussian_elimination(A, b):
    n = len(b)
    for i in range(n):
        max_row = i
        for j in range(i+1, n):
            if abs(A[j][i]) > abs(A[max_row][i]):
                max_row = j
        A[i], A[max_row] = A[max_row], A[i]
        b[i], b[max_row] = b[max_row], b[i]
        for j in range(i+1, n):
            factor = A[j][i] / A[i][i]
            for k in range(n+1):
                A[j][k] -= factor * A[i][k]
                b[j] -= factor * b[i]

    x = [0] * n
    for i in range(n-1, -1, -1):
        x[i] = (b[i] - sum(A[i][j] * x[j] for j in range(i+1, n))) / A[i][i]
    return x

coefficients = gaussian_elimination(A, b)

# Calculate the degree of the polynomial
degree = max(len([i for i, coeff in enumerate(coefficients) if coeff != 0]), 1)

return {
    "metric_name": "degree",
    "metric_value": degree,
    "instances_tested": 1,
    "conjecture_holds": degree >= math.sqrt(n),
    "counterexample": ""
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [random.randint(2, 1000003) for _ in range(30)]

    results = []
    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")
        results.append(trial_result)

    mean_value = sum(result["metric_value"] for result in results if result["metric_value"] is not None)
    std_value = math.sqrt(sum((result["metric_value"] - mean_value) ** 2 for result in results if result["metric_value"] is not None) / len(results))
    support_fraction = sum(result["conjecture_holds"] for result in results) / len(results)

    if all(result["conjecture_holds"] for result in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
    else:
        first_failing_seed = next((result["seed"] for result in results if not result["conjecture_holds"]))
        print(f"RESULT: FALSIFIED counterexample='degree_too_low' first_failing_seed={first_failing_seed}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

TIMEOUT after 240s

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Test timed out before producing results, preventing validation of support fraction or counterexample detection. | next: Retry test with extended timeout and debug resource constraints

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	51343
2	propose	ollama_remote	qwen3:8b	0	0	88794
3	preregistration	ollama_remote	qwen3:8b	0	0	27622
4	novelty	ollama_remote	qwen3:8b	0	0	22399
5	novelty	ollama_remote	qwen3:8b	0	0	24680
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12551
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	6202
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10872
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9933
10	judge	ollama_remote	qwen3:8b	0	0	126044

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 380440 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in `research/audit/4d43beaa3b30.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/4d43beaa3b30.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/4d43beaa3b30.tar.gz (if generated)